

PubSub Networking Algorithms

ARC PubSub Team

May 14, 2026

Disclaimer: This document is a work in progress. Do not take anything written here as the ground truth.

1 Introduction

This document overviews the algorithms run by the different layers of the PubSub mechanism proposed in [1]. The algorithms are not aimed at being fully precise, but should still give enough detail to be able to reason concretely about security and efficiency of the system.

2 Preliminaries

2.1 Cryptographic Building Blocks

The systems we describe in the sequel rely on digital signatures. A digital signature scheme Sig is a tuple $(\text{Setup}, \text{Keygen}, \text{Sign}, \text{Verify})$ defined as follows:

param $\leftarrow \text{Sig.Setup}(1^\kappa)$. Given a security parameter 1^κ , produces system parameters **param**.

(vk, sk) $\leftarrow \text{Sig.Keygen}(\text{param})$. Given system parameters **param**, generates a signing key **sk** and matching verification key **vk**.

$\sigma \leftarrow \text{Sig.Sign}(\text{sk}, \mathbf{m})$. Given a message **m** and a signing key **sk**, produces a signature σ .

$\top/\perp \leftarrow \text{Sig.Verify}(\text{vk}, \sigma, \mathbf{m})$. Given a verification key **vk**, signature σ , and message **m**, returns $\top$ if σ is a valid signature for **vk** and **m**, or $\perp$ otherwise.

2.2 Peer-to-Peer Systems

In Section 3, Section 4 and Section 5 we abstract different layers of a peer-to-peer PubSub scheme as a system in which all nodes implement the following

interface for each layer¹:

$\mathbf{st} \leftarrow \mathbf{Setup}(1^\kappa, \mathbf{aux})$. Given a security parameter 1^κ , and some application specific data $\mathbf{aux}$, returns an updated node state $\mathbf{st}$. We assume that the node state $\mathbf{st}$ is accessible by all other functions.

$\mathcal{D}_{out} \leftarrow \mathbf{Select}(\mathcal{D}, \mathcal{D}_{in}, k)$. Given a node descriptor $\mathcal{D}$, an input set of node descriptors $\mathcal{D}_{in}$ and a target size k , returns a set $\mathcal{D}_{out}$ of at most k descriptors chosen among $\mathcal{D}_{in}$.

$\top/\perp \leftarrow \mathbf{Send}(\mathcal{D})$ Picks the set of descriptors $\mathcal{D}$ to send to party identified by $\mathcal{D}$, and shares them.

$\top/\perp \leftarrow \mathbf{Recv}(\mathcal{D}, \mathcal{D}_{in})$. Processes the set of descriptors $\mathcal{D}_{in}$ received from party identified by $\mathcal{D}$.

$\top/\perp \leftarrow \mathbf{Run}(\mathbf{aux})$. Given some application specific data $\mathbf{aux}$, it sets up the node and enters an infinite loop for gossiping with peers.

When describing the behavior of peers in a peer-to-peer system, we assume a generic interface for sending and receiving messages via some chosen network (based, e.g., on the TCP/IP stack). We use $\mathbf{net.Send}(\mathbf{netaddr}, \mathbf{msg})$ to denote sending $\mathbf{msg}$ to the address specified by $\mathbf{netaddr}$, and $\mathbf{msg} \leftarrow \mathbf{net.Recv}(\mathbf{pattern})$ to denote the $\mathbf{msg}$ received from listening at the network address defined by $\mathbf{pattern}$. In the following, we use node and peer as synonyms.

2.3 Topic Registry

In the context of Cardano PubSub, we follow [1] in assuming the existence of a smart contract that maintains a registry of exiting topics, **TopicReg**. In the following, we assume that the topic registry is a map indexed by $\mathbf{tid}$ (a unique 256 bit string). For each $\mathbf{tid}$ entry, the registry stores the topic name (**tname**), topic owners (**owns**), nodes that can publish on the topic (**pubs**), retention period (**P**), and replication factor (**F**). $|\mathbf{TopicReg}|$ denotes the number of entries in the topic registry.

3 Peer Sampling Layer

The peer sampling layer is aimed at providing a node with a "base" set of peers. This set of peers will be fed into the navigation and dissemination layers. As specified in [1], we consider SecureCyclon as the peer-to-peer system for peer sampling. We define SecureCyclon's **Setup**, **Select**, **Send**, and **Recv** algorithms next, following [2] for reference.

¹Note that these are not pure functions, as they depend (read and write) on internal state. This approach is followed here as the three layers interact with each other, and facilitates the pseudocode. It may not be the best option for implementation, though.

In SecureCyclon, node descriptors have two main "parts", which we call here "base descriptor", and "chain of ownership", and use "full descriptor" to denote the combination. That is, a full descriptor is a tuple $(\mathcal{D}, [(vk_1, \sigma_1), \dots, (vk_n, \sigma_n)])$, where $\mathcal{D}$ is the base descriptor, and $[(vk_1, \sigma_1), \dots, (vk_n, \sigma_n)]$ is the chain of ownership. Note that the chain of ownership is iteratively formed, and each new signature σ covers the base descriptor as well as the previous pairs in the list. Given a full descriptor $\hat{\mathcal{D}}$, we use $\mathcal{D} \leftarrow \text{Base}(\hat{\mathcal{D}})$ to denote a helper function that returns the base descriptor, and $[(vk_1, \sigma_1), \dots, (vk_n, \sigma_n)] \leftarrow \text{Chain}(\hat{\mathcal{D}})$, for $n \geq 1$ to denote a helper function that returns the chain of ownership.

Algorithm 1 PS.Setup

Input:

- 1^κ The security parameter.
- aux Any additional auxiliary data.

Output:

- st Updated node state, with structure $(\mathcal{D}, sk, l, V)$.
- $\mathcal{D}$ The produced node descriptor.

```

(name, netaddr,  $\Delta$ ,  $l$ )  $\leftarrow$  aux
(vk, sk)  $\leftarrow$  Sig.Setup( $1^\kappa$ )
 $id \leftarrow$  Encode(Hash(name, netaddr, vk)) // Note: This choice is arbitrary
t  $\leftarrow$  GetTime()
 $\mathcal{D} \leftarrow (id, netaddr, vk, t)$ 
st.PS  $\leftarrow (\mathcal{D}, sk, \Delta, l, \emptyset)$  // Last  $\emptyset$  is the view
return st

```

Algorithm 2 PS.Select

Input:

- $\mathcal{D}$ The reference node's descriptor (may not be the caller.)
- $\mathcal{D}_{in}$ A set of node descriptors.
- k The number of node descriptors to select and transfer ownership of.

Output:

- $\mathcal{D}_{out}$ The set of selected descriptors, at most of size k .

```
( $\mathcal{D}_{self}, sk, \cdot, \cdot, \cdot$ )  $\leftarrow$  st.PS
 $\mathcal{D}_{out} \leftarrow \emptyset$ 
// Prepare a fresh descriptor of self // TODO: Should this only be done
when sending (not when receiving)?
 $t \leftarrow \text{GetTime}()$ 
 $\hat{\mathcal{D}}_{self} \leftarrow (\mathcal{D}_{self}.id, \mathcal{D}_{self}.netaddr, \mathcal{D}_{self}.vk, t)$ 
 $\sigma_{\mathcal{D}} \leftarrow \text{Sign}(sk, (\hat{\mathcal{D}}_{self}, \mathcal{D}.vk))$ 
 $\mathcal{D}_{out} \stackrel{\cup}{\leftarrow} \{(\mathcal{D}_{self}, [(\mathcal{D}.vk, \sigma_{\mathcal{D}})])\}$ 
// Fetch and prepare transfer of  $k - 1$  random descriptors from local view
for  $i \in [k - 1]$  do
     $\hat{\mathcal{D}}_i \stackrel{\$}{\leftarrow} \mathcal{D}_{in}, \mathcal{D}_i \leftarrow \text{Base}(\hat{\mathcal{D}}_i), chain_i \leftarrow \text{Chain}(\hat{\mathcal{D}}_i)$ 
     $\sigma_{\mathcal{D}_i} \leftarrow \text{Sign}(sk, (\hat{\mathcal{D}}_i, \mathcal{D}.vk))$  // Note:  $\sigma_{\mathcal{D}_i}$  also covers prior signatures in  $\hat{\mathcal{D}}_i$ 
     $\mathcal{D}_{out} \stackrel{\cup}{\leftarrow} \{(\mathcal{D}_i, [chain_i, (\mathcal{D}.vk, \sigma_{\mathcal{D}_i})])\}$ 
return  $\mathcal{D}_{out}$ 
```

Algorithm 3 PS.Send

Input:

- $\mathcal{D}$ the target descriptor.

Output:

- $\top$ if all went well, $\perp$ otherwise.

```
( $\cdot, \cdot, \cdot, l, V$ )  $\leftarrow$  st.PS
if  $\text{Base}(\mathcal{D}) \notin V$  then // Sender must own a descriptor created by receiver
    return  $\perp$ 
 $\mathcal{D} \leftarrow \text{PS.Select}(\mathcal{D}, V, l)$ 
 $\text{net.Send}(\mathcal{D}.netaddr, \mathcal{D})$ 
 $\text{st.PS}.V \leftarrow \text{st.PS}.V \setminus \mathcal{D}$  // Register ownership transfer [Jesus: this abuses
notation, as  $\mathcal{D}$  contains extended chains ( $\setminus$  only "looks at" base descriptors)]
// TODO:  $\mathcal{D}$  must also be removed from the view!
return  $\top$ 
```

Algorithm 4 PS.Recv

Input:

- $\mathcal{D}$ the descriptor of the calling party.
- $\mathcal{D}_{in}$ the received set of descriptors (sent by some other party).

Output:

- $\top$ if ok, $\perp$ otherwise.

```
( $\mathcal{D}_{self}, sk, \Delta, l, V$ )  $\leftarrow$  st.PS
// Sender must own a descriptor created by receiver
if Base( $\mathcal{D}_{self}$ )  $\notin \mathcal{D}_{in}$  then
    return  $\perp$ 
// Frequency check
for  $id \in GetNodesIDs(\mathcal{D}_{in})$  do
     $\mathcal{D}_{id} \leftarrow GetDescriptorsFromNodeID(id, \mathcal{D}_{in})$ 
    if  $\exists \mathcal{D}, \mathcal{D}' \in \mathcal{D}_{id}$  st  $|\mathcal{D}.t - \mathcal{D}'.t| < \Delta$  then
        return  $\perp$ 
// Ownership check
for  $\mathcal{D}' \in \mathcal{D}_{in}$  do
    if Base( $\mathcal{D}'$ )  $\in$  Base(st.PS.V) then // [Jesus: Base(st.PS.V) overloads notation]
        Let  $\mathcal{D}''$  be the full descriptor matching Base( $\mathcal{D}'$ ) in st.PS.V
        if Chain( $\mathcal{D}'$ )  $\not\subseteq$  Chain( $\mathcal{D}''$ )  $\wedge$  Chain( $\mathcal{D}''$ )  $\not\subseteq$  Chain( $\mathcal{D}'$ ) then
            return  $\perp$ 
    if VerifyChain(Chain( $\mathcal{D}'$ )) = false then
        return  $\perp$ 
 $\mathcal{D}_{out} \leftarrow$  PS.Select( $\mathcal{D}, V, l$ ) // Pick new descriptors for the calling node
net.Send( $\mathcal{D}.netaddr, \mathcal{D}_{out}$ )
st.PS.V  $\leftarrow$  (st.PS.V  $\setminus \mathcal{D}_{out}$ )  $\cup \mathcal{D}_{in}$  // Update view
return  $\top$ 
```

Algorithm 5 PS.Run

Input:

- 1^κ , security parameter
- `name`, a chosen application name
- `netaddr`, the public IP address and port number for the node to run
- l , target view length (nodes in view)
- Δ , period to wait for gossiping
- $\mathcal{D}_{btp}$, a bootstrapping node (may be extended to a set)

Output:

- Nothing. The program runs until stopped.

```
st  $\leftarrow$  PS.Setup( $1^\kappa$ , (name, netaddr,  $\Delta$ ,  $l$ )
// Bootstrapping, trusted nodes are configured to respond peers not owning
their descriptors
PS.Send( $\mathcal{D}_{btp}$ ) // Signals existence
while true do
    // Receive and send can run concurrently (but view must be mutexed?)
    (
        (( $\mathcal{D}$ ,  $\mathcal{D}$ )  $\leftarrow$  net.Recv(*); PS.Recv( $\mathcal{D}$ ,  $\mathcal{D}$ ))
        |
        ( $\mathcal{D}_{snd} \xleftarrow{\$}$  st.PS.V; PS.Send( $\mathcal{D}_{snd}$ )) // TODO:  $\mathcal{D}_{snd}$  is not picked at
        random - it must be the oldest peer.
    )
end while
```

4 Navigation Layer

The navigation layer builds on top of the peer-sampling layer, building a structure that prioritizes the establishment of connections between nodes interested in similar topics. As specified in [1], the chosen peer-to-peer scheme for the navigation layer is Vicinity [3].

Algorithm 6 Nav.Setup

Input:

- 1^κ The security parameter.
- `aux` Any additional auxiliary data.

Output:

- `st` Updated node state.

```
( $b, c$ )  $\leftarrow$  aux
st.Nav  $\leftarrow$  ( $b, c, \emptyset$ )
return st
```

Algorithm 7 Nav.Select. Navigation selection algorithm, approximating the caller's outgoing links in the target structure

Input:

- $\mathcal{D}$ The reference node's descriptor (may not be the caller.)
- $\mathcal{D}_{in}$ A set of node descriptors.
- k The number of node descriptors to select per finger.

Output:

- $\mathcal{D}_{out}$ The calling peer's new (proposed) view.

```

( $b, \cdot, \cdot$ )  $\leftarrow$  st.Nav
 $\mathcal{D}_{out} \leftarrow \emptyset$ 
// Get topics that  $\mathcal{D}$  is subscribed to TODO: How is this done?
tids  $\leftarrow$  GetSubscriptions( $\mathcal{D}$ )
// For each subscribed topic, iterate through its  $b^i$  distant finger topics
for  $t \in$  tids do
     $\mathcal{D}_{out} \leftarrow \bigcup FindClosestFreshestPeersPerTopic(\mathcal{D}_{in}, t, k)$ 
    for  $i \in [0, \lfloor \log_b(|TopicReg|/2) \rfloor + 1]$  do
        // Find  $c$  subscribers as close to tid  $t \pm b^i$  as possible
         $\mathcal{D}_{out} \leftarrow \bigcup FindClosestFreshestPeersPerTopic(\mathcal{D}_{in}, t + b^i, k)$ 
         $\mathcal{D}_{out} \leftarrow \bigcup FindClosestFreshestPeersPerTopic(\mathcal{D}_{in}, t - b^i, k)$ 
return  $\mathcal{D}_{out}$  //  $\mathcal{D}_{out}$  has size  $(2 * (\lfloor \log_b(|TopicReg|/2) \rfloor + 1) + 1) * k * |tids|$ 

```

Algorithm 8 Nav.Send

Input:

- $\mathcal{D}$ the target descriptor.

Output:

- $\top$ if ok, $\perp$ otherwise.

```

( $\cdot, \cdot, V$ )  $\leftarrow$  st.Nav
 $\mathcal{D} \leftarrow$  Nav.Select( $\mathcal{D}, V, 1$ ) // TODO: [Jesus: Should  $V$  here be set to the PS view?
If not, how and when do we feed PS nodes into Nav?]
net.Send( $\mathcal{D}.netaddr, \mathcal{D}$ )
return  $\top$ 

```

Algorithm 9 Nav.Recv

Input:

- $\mathcal{D}$ the descriptor of the calling party.
- $\mathcal{D}_{in}$ the received set of descriptors (sent by some other party).

Output:

- $\top$ if ok, $\perp$ otherwise.

```
( $\cdot, c, \mathbf{V}$ )  $\leftarrow$  st.Nav  
 $\mathcal{D}_{out} \leftarrow$  Nav.Select( $\mathcal{D}, \mathbf{V} \cup \mathcal{D}_{in}, c$ )  
st.Nav. $\mathbf{V} \leftarrow \mathcal{D}_{out}$   
return  $\mathcal{D}_{out}$ 
```

Algorithm 10 Nav.Run

Input:

- b , the routing base.
- c , the number of nodes per finger.

Output:

- Nothing. The program runs until stopped.

```
st  $\leftarrow$  Nav.Setup( $b, c$ )  
while true do  
  // Receive and send can run concurrently (but view must be mutexed?)  
  (  
    (( $\mathcal{D}, \mathcal{D}$ )  $\leftarrow$  net.Recv(*); Nav.Recv( $\mathcal{D}, \mathcal{D}$ ))  
    |  
    ( $\mathcal{D}_{snd} \xleftarrow{\$}$  st.Nav. $\mathbf{V}$ ; Nav.Send( $\mathcal{D}_{snd}$ ))  
  )
```

5 Dissemination Layer

For each topic a node subscribes to, the dissemination layer connects it with peers subscribed to the same topic. This has the end goal of ensuring that any message that is published in any given topic is reliably delivered to all subscribers of the topic. For simplicity, the following algorithms are described *per topic*. Hence, we assume here that for each topic a node subscribes to, it needs to spawn a fresh dissemination layer for that topic – however, this choice is arbitrary, and the algorithms could be extended to handle multiple topics.

Algorithm 11 Dis.Setup

Input:

- 1^κ The security parameter.
- aux Any additional auxiliary data.

Output:

- st Updated node state.

$(tid, rlinks, k) \leftarrow aux$
 $st.Dis \leftarrow (tid, rlinks, k, \emptyset)$
return st

Algorithm 12 Dis.Select. Dissemination selection algorithm, approximating the caller's outgoing links in the target structure.

Input:

- $\mathcal{D}$ The reference node's descriptor.
- $\mathcal{D}_{in}$ A set of node descriptors.
- k The number of closest node descriptors to select.

Output:

- $\mathcal{D}_{out}$ The set of selected descriptors, at most of size k .

$(tid, rlinks, \cdot, \mathbf{V}) \leftarrow st.Dis$
 $\mathcal{D}_{out} \leftarrow \emptyset$
 $\mathcal{D}_{out} \leftarrow \bigcup FindClosestFreshestLowPeersInTopic(\mathcal{D}_{in}, st.PS.\mathcal{D}, k/2)$
 $\mathcal{D}_{out} \leftarrow \bigcup FindClosestFreshestHighPeersInTopic(\mathcal{D}_{in}, st.PS.\mathcal{D}, k/2)$
 $\mathcal{D}_{out} \leftarrow \bigcup FindRandomPeersPerTopic(st.Nav.\mathbf{V}, tid, rlinks)$
return $\mathcal{D}_{out}$ // Size is $(k + rlinks)$

Algorithm 13 Dis.Send

Input:

- $\mathcal{D}$ the target descriptor.

Output:

- $\top$ if ok, $\perp$ otherwise.

$(tid, \cdot, k, \mathbf{V}) \leftarrow st.Dis$
 $\mathcal{D} \leftarrow Dis.Select(\mathcal{D}, \mathbf{V} \cup \{st.Nav.\mathbf{V}[tid]\}, k)$
 $net.Send(\mathcal{D}.netaddr, \mathcal{D})$
return $\top$

Algorithm 14 Dis.Recv

Input:

- $\mathcal{D}$ the descriptor of the calling party.
- $\mathcal{D}_{in}$ the received set of descriptors (sent by some other party).

Output:

- $\top$ if ok, $\perp$ otherwise.

```
( $\cdot, \cdot, k, \mathbf{V}$ )  $\leftarrow$  st.Dis  
 $\mathcal{D}_{out} \leftarrow$  Dis.Select( $\mathcal{D}, \mathbf{V} \cup \mathcal{D}_{in}, k$ )  
st.Dis. $\mathbf{V} \leftarrow \mathcal{D}_{out}$   
return  $\top$ 
```

Algorithm 15 Dis.Run

Input:

- tid, the topic identifier that this Dissemination instance will manage.
- *rlinks*, the number of random links to connect to.
- k , the number of nodes per limb.

Output:

- Nothing. The program runs until stopped.

```
st  $\leftarrow$  Dis.Setup(tid, rlinks,  $k$ )  
while true do  
  // Receive and send can run concurrently (but view must be mutexed?)  
  (  
    (( $\mathcal{D}, \mathcal{D}$ )  $\leftarrow$  net.Recv(*); Dis.Recv( $\mathcal{D}, \mathcal{D}$ ))  
    |  
    ( $\mathcal{D}_{snd} \xleftarrow{\$}$  st.Nav. $\mathbf{V}$ ; Dis.Send( $\mathcal{D}_{snd}$ ))  
  )
```

6 Cardano PubSub

The architecture for the proposed Cardano PubSub system stacks the peer-to-peer systems in the previous sections, as follows:

- Layer 0: Peer-sampling layer Section 3, aimed at providing the running node with a good view of all the peers in the network.
- Layer 1: Navigation layer Section 4, aimed at facilitating nodes to connect to other nodes with "similar" interests.
- Layer 2: Dissemination layer Section 5, aimed at letting nodes reliably publish/receive messages into/from topics they belong/subscribe to.

Thus, informally, when a node starts, it has to run the three layers concurrently.

7 Informal Assumptions and Security Properties

Next, we give a (possibly incomplete) list of informal security notions that each layer is expected to achieve, as well as the assumptions that each layer relies on.

Security of Peer Sampling. The peer sampling layer is expected to contribute connections to random peers in the network. As such, the expectation is that, after some amount of time, all peers in the network have appeared in the view of a peer running PS.Run, perhaps with some lower bound in the probability of not doing so. *We assume that there is a (set of) bootstrapping nodes that are always accessible.*

Security of Navigation. The navigation layer is expected to allow any given peer to quickly find topics of its interest. Consequently, *provided that a peer has a good view of the peer-to-peer network* (provided by the peer sampling layer), there should be a time bound for the time it takes for any given peer to connect to some per of any chosen topic.

Security of Dissemination. The dissemination layer is expected to help distribute messages on any given topic, to all nodes subscribed to that topic. Therefore, *provided that a publishing node is already in contact with node of the same topic*, there should be a time bound for the time it takes for a message to be posted in some topic, and for this message to reach all (but a bounded number of) nodes subscribed to this topic.

References

- [1] Spyros Voulgaris Alexandros Antonov, Evangelos Kolyvas. Cardano Pub/Sub Framework Design and Architecture. Technical report, Athens University, September 2024. Accessed: 2026-04-17. URL: <https://drive.google.com/file/d/1oZiKWbW1mXgRbl8iYxsBzkapvSx1fVwF/view?usp=sharing>.
- [2] Alexandros Antonov and Spyros Voulgaris. Securecyclon: Dependable peer sampling. In *43rd IEEE International Conference on Distributed Computing Systems, ICDCS 2023, Hong Kong, July 18-21, 2023*, pages 1–12. IEEE, 2023. doi:10.1109/ICDCS57875.2023.00041.
- [3] Spyros Voulgaris and Maarten van Steen. VICINITY: A pinch of randomness brings out the structure. In David M. Eysers and Karsten Schwan,

editors, *Middleware 2013 - ACM/IFIP/USENIX 14th International Middleware Conference, Beijing, China, December 9-13, 2013, Proceedings*, Lecture Notes in Computer Science, pages 21–40. Springer, 2013. doi: 10.1007/978-3-642-45065-5_2.